

Homework 1: Functions

- Due: Thursday 09/03 @ 11:59pm
- Points: 1
- Download: hw01.zip

To receive credit, you must solve each problem and then complete a short checkoff interview about your solution. You may use Preceptor for the interview or come to office hours to be interviewed by a member of the course staff. Staff will be doing in-person checkoffs for up to 3 business days after the assignment deadline (including any approved extension). If a checkoff happens more than 3 business days after the regular deadline, staff may ask to confirm your extended deadline via Flexextensions. You will submit a Provenance bundle that includes a record of how you used VS Code, including interactions with Preceptor. Please do not use AI tools other than Preceptor for this assignment.

Readings:

- 1.1: Getting Started
- 1.2: Elements of Programming
- 1.3: Defining New Functions
- 1.4: Designing Functions
- 1.5: Control

Required Questions

Q1: A Plus Abs B

Python's operator module contains two-argument functions such as `add` and `sub` for Python's built-in arithmetic operators. For example, `add(2, 3)` evaluates to 5, just like the expression `2 + 3`.

Fill in the blanks in the following function to add `a` to the absolute value of `b`, without calling the `abs` function. You may not modify any of the provided code other than the two blanks.

```
def a_plus_abs_b(a, b):
    """Return a+abs(b), but without calling abs.

    >>> a_plus_abs_b(2, 3)
    5
    >>> a_plus_abs_b(2, -3)
    5
    >>> a_plus_abs_b(-1, 4)
    3
    >>> a_plus_abs_b(-1, -4)
    3
    """
    if b < 0:
        f = ____
    else:
        f = ____
    return f(a, b)

python3 -m pytest -k a_plus_abs_b
```

Q2: Two of Three

Write a function that takes three positive numbers as arguments and returns the sum of the squares of the two smallest numbers. Use only a single line for the body of the function.

```
def two_of_three(i, j, k):
    """Return m*m + n*n, where m and n are the two smallest members of the
    positive numbers i, j, and k.

    >>> two_of_three(1, 2, 3)
    5
    >>> two_of_three(5, 3, 1)
    10
```

```
>>> two_of_three(10, 2, 8)
68
>>> two_of_three(5, 5, 5)
50
"""
return _____
```

Hint: Consider using the max or min function:

```
>>> max(1, 2, 3)
3
>>> min(-1, -2, -3)
-3

python3 -m pytest -k two_of_three
```

Q3: Largest Factor

Write a function that takes an integer n that is greater than 1 and returns the largest integer that is smaller than n and evenly divides n .

```
def largest_factor(n):
    """Return the largest factor of n that is smaller than n.

    >>> largest_factor(15) # factors are 1, 3, 5
    5
    >>> largest_factor(80) # factors are 1, 2, 4, 5, 8, 10, 16, 20, 40
    40
    >>> largest_factor(13) # factors are 1, 13
    1
    """
    """ YOUR CODE HERE """
```

Hint: To check if b evenly divides a , use the expression $a \% b == 0$, which can be read as, "the remainder when dividing a by b is 0."

```
python3 -m pytest -k largest_factor
```

Q4: Hailstone

Douglas Hofstadter's Pulitzer-prize-winning book, *Gödel, Escher, Bach*, poses the following mathematical puzzle.

- Pick a positive integer n as the start.
- If n is even, divide it by 2.
- If n is odd, multiply it by 3 and add 1.
- Continue this process until n is 1.

The number n will travel up and down but eventually end at 1 (at least for all numbers that have ever been tried—nobody has ever proved that the sequence will terminate). Analogously, a hailstone travels up and down in the atmosphere before eventually landing on earth.

This sequence of values of n is often called a Hailstone sequence. Write a function that takes a single argument with formal parameter name n , prints out the hailstone sequence starting at n , and returns the number of steps in the sequence:

```
def hailstone(n):
    """Print the hailstone sequence starting at n and return its length.

    >>> a = hailstone(10)
    10
    5
    16
    8
    4
    2
    1
    >>> a
    7
    >>> b = hailstone(1)
    1
    >>> b
    1
```

```
"""
*** YOUR CODE HERE ***
"""
```

Hailstone sequences can get quite long! Try 27. What's the longest you can find?

Note that if $n == 1$ initially, then the sequence is one step long. Hint: If you see 4.0 but want just 4, try using floor division `//` instead of regular division `/`.

```
python3 -m pytest -k hailstone
```

Fun fact: In 2019, there was a major development in understanding how the hailstone conjecture works for most numbers! This 2026 StarTalk interview with Terence Tao discusses the problem.

Submit

Run Provenance: Prepare Submission Bundle from the VS Code command palette to create your submission zip, and upload that zip to Gradescope. For a refresher on how to do this, refer to Lab 00.